Linear Regression Review

Huu Loi Bui Vuong Khang Huynh

September 15, 2025

Contents

1 Linear Regression **2**
1.1 Introduction 2
1.2 Formulation 2
1.3 Linear Regression in Machine Learning 3
1.3.1 Problem Definition 3
1.3.2 Solving 4

2 Python Tools for Solving Linear Regression Models **7**
2.1 scikit-learn 7
2.2 statsmodel 9

1 Linear Regression

1.1 Introduction

Linear regression is a fundamental method in machine learning and statistics, used to model the relationship between independent variables and a dependent variable through a linear function. This method plays a crucial role in various fields such as economics (revenue forecasting), medicine (disease risk estimation), and engineering. In the field of machine learning, linear regression is a type of supervised learning algorithm that learns from labeled datasets and maps data points to the most optimal linear function, which can be used to make predictions on new datasets. The algorithm assumes a linear relationship between the input and output, meaning the output changes at a constant rate as the input changes. This relationship is represented by a straight line. In this report, we will detail the foundational concepts, types of regression, and methods for solving the problem, delving deeply into the mathematical formulas.

1.2 Formulation

Given a dataset $\{y_i,\ x_{i1},\ x_{i2}, \ldots,\ x_{im}\}_{i=1}^{n}$ consisting of n statistical units, the linear regression model describes the relationship between the dependent variable $\mathbf{y}$ and the vector of regressors $\mathbf{x}$. This relationship is linear and incorporates a disturbance term or error variable ε, an unobserved random variable that adds 'noise' to the relationship. Accordingly, the model is expressed as:

$$\hat{y}_i = w_0 + w_1 x_{i1} + \cdots + w_m x_{im} + \varepsilon_i = \mathbf{x}_i^{\mathsf{T}}\mathbf{w} + \varepsilon_i, \qquad i = 1, \ldots, n;\ \mathbf{w} \text{ is a weight vector; } \hat{\mathbf{y}} \text{ is the estima}$$

Typically, these n equations are combined and expressed in matrix form as:

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + \varepsilon,$$

where:

$$\hat{\mathbf{y}} = \begin{bmatrix} \hat{y_1} \\ \hat{y_2} \\ \vdots \\ \hat{y_n} \end{bmatrix}, \quad \mathbf{X} = \begin{bmatrix} \mathbf{x}_1^{\mathsf{T}} \\ \mathbf{x}_2^{\mathsf{T}} \\ \vdots \\ \mathbf{x}_n^{\mathsf{T}} \end{bmatrix} = \begin{bmatrix} 1 & x_{11} & \cdots & x_{1m} \\ 1 & x_{21} & \cdots & x_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & \cdots & x_{nm} \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_m \end{bmatrix}, \quad \varepsilon = \begin{bmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \vdots \\ \varepsilon_n \end{bmatrix}.$$

The model aims to obtain the optimal values for the weight vector w, which should minimize the sum of square of residuals.
Note that:

$$\bar{y} = \frac{y_1 + y_2 + ... + y_n}{n}$$

$$SSE = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2, SSR = \sum_{i=1}^{n}(\hat{y}_i - \bar{y})^2, SST = SSE + SSR = \sum_{i=1}^{n}(y_i - \bar{y})^2$$

Formally, the linear regression model searches for the weight matrix w that solves:

$$\min_{w} \frac{SSE}{n}$$

These formulations provide two metrics to evaluate the precision of the estimated linear regression model: Root Mean Square Error (RMSE) and the coefficient of determination

R^2. The Root Mean Square Error $RMSE = \sqrt{\frac{SSE}{n}})$ indicates the variation of each observed data y_i compared to its corresponding estimated value $\hat{y}_i$. An RMSE close to 0 shows that the model's prediction is close to the actual value of the dependent variable in the dataset. On the other hand, the coefficient of determination $R^2 = \frac{SSR}{SST}$ calculates the proportion of variation in the response variables that is explained by the different values of regressors compared to the total variation ($0 \leq R^2 \leq 1$). A value of R^2 close to 1 indicates that most of the variation of the response data is explained by the different values of regressors, meaning the relationship between $\mathbf{Y}$ and $\mathbf{x}$ is almost linear.

1.3 Linear Regression in Machine Learning

1.3.1 Problem Definition

The linear regression problem aims to find the weight vector w to approximate the optimal regression function $h(x)$ from input data. Inputs include:

- A set of N feature vectors $\mathbf{x}_n = [x_{n,0}, x_{n,1}, \ldots, x_{n,M-1}]^T$, $x_{n,0} = 1$, $\forall n \in [1, \ldots, N]$.
- Target vector $t = [t_1, t_2, \ldots, t_N]^T$, t_n is the continuous measured value for $\mathbf{x}_n$.

Data is organized in matrix form: $X = \begin{bmatrix} 1 & x_{1,1} & \ldots & x_{1,M-1} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{N,1} & \ldots & x_{N,M-1} \end{bmatrix}$, $t = \begin{bmatrix} t_1 \\ \vdots \\ t_N \end{bmatrix}$.

Assumption:

$$t = h(\mathbf{x}) + \epsilon \tag{1}$$

- $h(\mathbf{x})$ is the unknown function (underlying true function of the data), which can be approximated by a linear function $\hat{y}(x, w)$, see Equation (2)
- $\epsilon \sim \mathcal{N}(0, \sigma^2)$ is Gaussian noise, mean 0, variance $\sigma^2 = \beta^{-1}(precision)$.
- Data points follow the i.i.d. principle (independent and identically distributed).

The objective of the problem is to find the weight vector w for the hypothesis function $\hat{y}(x, w)$, which helps to construct an optimal linear prediction model under the Gaussian noise assumption, such that the likelihood of the data is maximized (i.e., the error is minimized).

Linear Regression Model:

The basis of linear regression and more broadly, supervised learning is a linear prediction function $\hat{y}(x, w)$, used to approximate the unknown function $h(x)$ that generates the true data.

$$\begin{aligned} \hat{y}(\mathbf{x}, \mathbf{w}) &= w_0 x_0 + w_1 x_1 + \cdots + w_{M-1} x_{M-1} \\ &= \sum_{m=0}^{M-1} w_m x_m \\ &= \mathbf{w}^\mathsf{T} \mathbf{x} \end{aligned} \tag{2}$$

The goal of the problem is to find the optimal weights $w = [w_0,\ w_1, \ldots,\ w_m]^\mathsf{T}$ so that $\hat{y}(\mathbf{x}, \mathbf{w})$ approximates the target t as closely as possible (by approximating $h(\mathbf{x})$).

1.3.2 Solving

From (1) and (2), we have:

- $E(t) = E(h(\mathbf{x})) + E(\epsilon) = E(h(\mathbf{x})) = \hat{y}(\mathbf{x}, \mathbf{w})$
- $Var(t) = Var(\epsilon) = \beta^{-1}$

Hence:

$$\begin{aligned} t &\sim \mathcal{N}\left(\hat{y}(\mathbf{x}, \mathbf{w}), \beta^{-1}\right) \\ &\stackrel{\text{def}}{=} p(t|\mathbf{x}, \mathbf{w}, \beta) \\ &= \frac{\beta^{\frac{1}{2}}}{(2\pi)^{\frac{1}{2}}} exp\left(-\frac{\beta}{2}[t - \hat{y}(\mathbf{x}, \mathbf{w})]^2\right) \end{aligned} \tag{3}$$

- $p(t|\mathbf{x}, \mathbf{w}, \beta)$ is the likelihood function, representing the probability of observing t given $\mathbf{x}$, $\mathbf{w}$, and β.

In this case, the target vector t is assumed to represent the simultaneous realization of all its component values $t_n, \forall n \in [1, \ldots, N]$.

$$p(t|\mathbf{X}, \mathbf{w}, \beta) = \prod_{n=1}^{N} p(t_n|\mathbf{x}, \mathbf{w}, \beta)$$

Our objective is to find the values of $\mathbf{w}$ and β that maximize $p(t|\mathbf{X}, \mathbf{w}, \beta)$.

To solve this problem, we use the maximum likelihood principle, leading to minimizing the negative log-likelihood (NLL) function:

$$\begin{aligned}\mathcal{L}(\mathbf{w}, \beta) &\stackrel{\text{def}}{=} -\ln\left(p(t|\mathbf{X}, \mathbf{w}, \beta)\right) \\ &= -\sum_{n=1}^{M} \ln\left(p(t_n|\mathbf{X}, \mathbf{w}, \beta)\right) \\ &\stackrel{(3)}{=} \beta E_D(\mathbf{w}) - \frac{N}{2}\ln\beta + \frac{N}{2}\ln(2\pi),\end{aligned}$$

with the sum-of-squares error function: $E_D(\mathbf{w}) = \frac{1}{2}\sum_{n=1}^{N}(t_n - \mathbf{w}^\mathsf{T}\mathbf{x}_n)^2$

This function is directly linked to the hypothesis, as the error is $t_n - \hat{y}(\mathbf{x}_n, \mathbf{w})$. Optimize with respect to $\mathbf{w}$ first (minimize $E_D(\mathbf{w})$), then β (the function $\mathcal{L}(\mathbf{w}, \beta)$ is convex with respect to each variable, $\mathbf{w}$ and β).

Derivative of $E_D(\mathbf{w})$ with respect to $\mathbf{w}$:

We have:

$$\begin{aligned}\frac{\partial}{\partial \mathbf{w}}\left(t_n - \mathbf{w}^\mathsf{T}\mathbf{x}_n\right)^2 &= 2\left(t_n - \mathbf{w}^\mathsf{T}\mathbf{x}_n\right)\cdot\frac{\partial}{\partial \mathbf{w}}\left(t_n - \mathbf{w}^\mathsf{T}\mathbf{x}_n\right) \\ &= -2\left(t_n - \mathbf{w}^\mathsf{T}\mathbf{x}_n\right)\cdot\frac{\partial}{\partial \mathbf{w}}\left(\mathbf{w}^\mathsf{T}\mathbf{x}_n\right) \\ &= -2\left(t_n - \mathbf{w}^\mathsf{T}\mathbf{x}_n\right)\cdot\frac{\partial}{\partial \mathbf{w}}\left(\mathbf{x}_n^\mathsf{T}\mathbf{w}\right) \\ &= 2\left(\mathbf{w}^\mathsf{T}\mathbf{x}_n - t_n\right)\cdot\mathbf{x}_n^\mathsf{T}\end{aligned}$$

Hence:

$$\frac{\partial E_D(\mathbf{w})}{\partial \mathbf{w}} = \sum_{n=1}^{N}(\mathbf{w}^\mathsf{T}\mathbf{x}_n - t_n)\cdot\mathbf{x}_n^\mathsf{T}$$

Solving the equation $\frac{\partial E_D(\mathbf{w})}{\partial \mathbf{w}} = 0^\mathsf{T}$:

$$\begin{aligned}\mathbf{w}^\mathsf{T}\sum_{n=1}^{N}\mathbf{x}_n\mathbf{x}_n^\mathsf{T} - \sum_{n=1}^{N} t_n\mathbf{x}_n^\mathsf{T} &= 0^\mathsf{T} \\ \mathbf{w}^\mathsf{T}\mathbf{X}^\mathsf{T}\mathbf{X} &= t^\mathsf{T}\mathbf{X} \\ \mathbf{X}^\mathsf{T}\mathbf{X}\mathbf{w} &= \mathbf{X}^\mathsf{T}t\end{aligned}$$

If $\mathbf{X}^\mathsf{T}\mathbf{X}$ is invertible, the solution for $\mathbf{w}$ is:

$$\mathbf{w}_{ML} = \left(\mathbf{X}^\mathsf{T}\mathbf{X}\right)^{-1}\cdot\mathbf{X}^\mathsf{T}t$$

Derivative of $\mathcal{L}(\mathbf{w}_{ML}, \beta)$ with respect to β:

$$\frac{\partial}{\partial \beta}\left(\beta E_D(\mathbf{w}_{ML}) - \frac{N}{2}\ln\beta\right) = E_D(\mathbf{w}_{ML}) - \frac{N}{2\beta}$$

Solving the equation $E_D(\mathbf{w}_{ML}) - \frac{N}{2\beta} = 0$ we have:

$$\beta_{ML}^{-1} = \frac{1}{N}\sum_{n=1}^{N}(t_n - \mathbf{w}_{\mathbf{ML}}^{\mathsf{T}}\mathbf{x}_n)^2$$

At that point, $\sigma^2 = \beta_{ML}^{-1}$ represents the variance of the noise estimated on the training set, and it needs to be re-evaluated on the test set for validation.

Iterative algorithm (mini-batch gradient descent): Using the average error function

With B data points sampled from N samples, the mean squared error function is defined as:

$$\begin{aligned}\bar{E}_D(w) &\stackrel{\text{def}}{=} \frac{1}{B}E_D(\mathbf{w}) \\ &= \frac{1}{2B}\sum_{n=1}^{B}(t_n - \mathbf{w}^T\mathbf{x}_n)^2 \end{aligned} \tag{4}$$

This function is used instead of $E_D(\mathbf{w})$ so that the error does not increase with the number of samples.

The average derivative of the error function according to Equation (4) is:

$$\Delta W_B = \frac{1}{B}\sum_{n=1}^{B}\left(\mathbf{w}^{(t-1)T}\mathbf{x}_n - t_n\right)\mathbf{x}_n \tag{5}$$

- $\mathbf{w}^{(t-1)}$: coefficient vector of the model at the previous iteration $(t-1)$.
- $\mathbf{w}^{(t-1)T}$: transpose of the vector $\mathbf{w}^{(t-1)}$.

The B data points can be sampled with replacement from the training set, or they can arrive sequentially (*online*).

Algorithm: Error minimization by gradient descent

1. Initialize $t = 0$; randomly generate (commonly from a normal distribution with sufficiently small values around 0) an initial value for $\mathbf{w}^t$; assign a sufficiently large value to e^t.

2. **Do:**

   (a) Update iteration index: $t = t + 1$; sample B data points from the training set.

   (b) Compute the error e^t using Eq.(4)

   (c) **If** $\|e^t - e^{(t-1)}\|$ is sufficiently small, then:

   - Trigger the stopping condition.

   (d) **Else:**

   - Compute the average derivative according to Eq.(5).
   - Update the coefficient vector:

   $$\mathbf{w}^t = \mathbf{w}^{(t-1)} - \eta \times \Delta W_B$$

   where η is the learning rate.

3. **While** the stopping condition is not satisfied.

4. Return the current solution $\mathbf{w}$.

2 Python Tools for Solving Linear Regression Models

Python provides a variety of libraries for building and evaluating linear regression models. This section reviews two particularly important libraries: **scikit-learn** and **statsmodels**, and their common methods to solve a simple linear regression problem as followed:

The dataset "All_GPUs.csv" provides a comprehensive collection of technical specifications for graphics processing units (GPUs) from manufacturers such as Nvidia, AMD, and Intel. It encompasses both consumer-grade and professional-grade GPUs released over several years. The regression model aims to find the linear relationship between dependent variable **Memory_Bandwidth** and regressors **Memory_Bus**, **Memory_Speed**, **ROPs** and **Memory_Type**.
The complete implementation, including data preprocessing, visualization, and regression modeling in Python is available here.

2.1 scikit-learn

The **scikit-learn** library is the most commonly used machine learning toolkit in Python. Its **LinearRegression** class supports training ordinary least squares Linear Regression models (to minimize the residual sum of squares between the observed targets and the targets predicted by approximation, without error variables) [3].

```
class sklearn.linear_model.LinearRegression(*, fit_intercept=True
    , copy_X=True, tol=1e-06, n_jobs=None, positive=False)
```

The parameters include:

- **fit_intercept** (bool, default = True): to decide whether to calculate the intercept w_0.
- **copy_X** (bool, default = True): to decide whether to copy the original training set (if True), or to overwrite it (if False).
- **tol** (float, default = 1e-6): specifies a different convergence criterion for the **scipy.sparse.linalg.ls** solver ($atol = btol = tol$). Let $r = y - Xw$ be the residual vector for the current approximate solution w. If $Xw = y$ seems to be consistent, lsqr terminates when $norm(r) <= atol * norm(X) * norm(w) + btol * norm(y)$. Otherwise, lsqr terminates when $norm(X^H r) <= atol * norm(X) * norm(r)$ [4]. This parameter is only necessary when X is sparse.
- **n_jobs** (int, default = None): the number of jobs for computation, which provide a speedup for sufficiently large problem (when X is sparse, the number of dependent variables $n_targets > 1$ or positive = True). The default case often means there is only one dependent variable.
- **positive** (bool, default = False): determine whether all coefficients w_i must be positive (if True).

The class' attributes consists of:

- **coef_** (array) stores the w matrix of coefficients, is of shape (m,) or (n, m) (where n is the number of samples and m is the number of regressors).
- **rank_** (int) represents the rank of matrix X when X is dense.
- **singular_** (1D array) stores singular values of X.
- **intercept_** (float) represents the intercept of y, is 0.0 when **fit_intercept** = False.
- **n_features_in_** (int) is the number of features.
- **feature_names_in_** (array) includes names of features (when X has feature names of type string), shape (n_features_in_,)

The class solely uses method **fit** to train its regression model:

```
fit(X, y, sample\_weight=None)
```

Parameter X and y represent the training set and target values, while sample_weight represents individual weights for each sample.
In the problem mentioned above, we implement the **LinearRegression** class with default parameters for a simple linear regression model (as the dataset is relatively small).

```
#! === 11. Linear regression model ===
model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

In addition, **scikit-learn** also provide other supporting methods for linear regression models, one of which being **train_test_split** for seperating the training set from the dataset.

```
#! === 10. Train/test split ===
X = main_data[["Memory_Bus", "Memory_Speed", "ROPs", "
    Memory_Type_Numeric"]]
y = main_data["Memory_Bandwidth"]
X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.3, random_state=0)
```

In this code section, the method split 30% of the regressors set X and dependent variable matrix y for the testing matrix and the remaining 70% for the training set.
Other essential methods include **mean_squared_error** and **r2_score**, which calculate the $RMSE$ and R^2 (coefficient of determination), from the input testing set **y_test** of the dependent variable, and the **predictions** matrix, to evaluate the precision of the estimation.

```
#! === 13. Evaluation ===
rmse = np.sqrt(mean_squared_error(y_test, predictions))
r2 = r2_score(y_test, predictions)
print(f"RMSE: {rmse}")
print(f"R-squared: {r2}")
```

The result of this code section is provided below:

```
Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
RMSE: 0.2717881540476916
R-squared: 0.9273096734616258
```

Figure 1: Errors of Linear Regression estimation by scikit-learn

2.2 statsmodel

Unlike **scikit-learn**, which focuses on prediction pipelines and model accuracy, **statsmodels** provides a toolkit specifically statistical estimation and inference. It implements ordinary least squares and a range of other estimation methods and returns not only point estimates for parameters but also their standard errors, t-statistics, p-values and confidence intervals, enabling formal hypothesis testing.
statsmodels also provides class **OLS** implementing an Ordinary Least Square model [5]:

```
class statsmodels.regression.linear_model.OLS(endog, exog=None,
    missing='none', hasconst=None, **kwargs)
```

The parameters include:

- **endog**: 1D array of dependent variable y
- **exog**: (n,m) shaped array of regressors X

- **missing**: of type string, whose available options are 'none' (default, no nan checking), 'drop' (dropping observation with nans) and 'raise' (raising error when nans exist). In this problem, no nan checking is required, as most nan values are removed during the preprocessing section.
- **hasconst**: of type bool, or None as default, indicates whether Xw includes a user-supplied constant, which there is none in this problem.
- ****kwargs**: extra arguments to set model's properties.

While **scikit-learn's LinearRegression** automatically calculates the intercept (for **fit_intercept** = True), **OLS** requires the input regressors matrix X a column of 1s indicating the coefficient of freedom w_0. **statsmodels** also provide the method **add_constant** for this purpose.

```
#! === 12. Statsmodels regression summary ===
X_train_const = sm.add_constant(X_train)
ols_model = sm.OLS(y_train, X_train_const).fit()
print(ols_model.summary())
```

The code section above yields the following result:

```
                            OLS Regression Results
==============================================================================
Dep. Variable:       Memory_Bandwidth   R-squared:
          0.921
Model:                            OLS   Adj. R-squared:                  0.921
Method:                 Least Squares   F-statistic:
          5244.
Date:                Mon, 15 Sep 2025   Prob (F-statistic):               0.00
Time:                        19:38:59   Log-Likelihood:                -343.37
No. Observations:                1800   AIC:
          696.7
Df Residuals:                    1795   BIC:
          724.2
Df Model:                           4

Covariance Type:            nonrobust
==========================================================================================
                         coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------------------
const                  -7.6193      0.234    -32.577      0.000      -8.078      -7.161
Memory_Bus              0.5787      0.020     29.543      0.000       0.540       0.617
Memory_Speed            1.0307      0.030     34.768      0.000       0.973       1.089
ROPs                    0.5738      0.016     36.955      0.000       0.543       0.604
Memory_Type_Numeric     0.0429      0.007      6.062      0.000       0.029       0.057
==============================================================================
Omnibus:                       70.515   Durbin-Watson:
          1.983
Prob(Omnibus):                  0.000   Jarque-Bera (JB):              138.982
Skew:                           0.271   Prob(JB):
       6.61e-31
==============================================================================
```

Figure 2: Result for statsmodels summary

Note that the R^2 calculated by **statsmodels' OLS** is relatively close to that by **scikit-learn's LinearRegression**, with a difference of approximately 0.0063. This indicates two models are equivalent in terms of precision.

References

[1] Wikipedia, "Linear regression", 2025. [Online]. Available: https://en.wikipedia.org/wiki/Linear_regression#References

[2] Le Thanh Sach - HCMUT, *"MO HINH TUYEN TINH, BAI TOAN HOI QUY"*, 2019.

[3] scikit-learn, "sklearn.linear_model.LinearRegression", 2025. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html

[4] SciPy, "scipy.sparse.linalg.lsqr", 2025. [Online]. Available: https://docs.scipy.org/doc/scipy-1.16.2/reference/generated/scipy.sparse.linalg.lsqr.html

[5] Statsmodels, "statsmodels.regression.linear_model.OLS", 2025. [Online]. Available: https://www.statsmodels.org/stable/generated/statsmodels.regression.linear_model.OLS.html